

CSC173: Project 2

Recursive Descent and Table-Driven Parsing

The goal of this project is to demonstrate your understanding of the formal model of context-free grammars and parsing by applying the principles of the model to a specific grammar (details below).

The goal of the project is **NOT** simply for you to write a program that parses strings from language defined by the specific grammar described in this document. The goal is for you to demonstrate how the formal model allows you build a parser based on the grammar **mechanically** with almost no thinking required.

Process not product.

Given a parsable grammar, for this project you must first construct a recursive-descent parser for the grammar using parsing functions, as seen in class and in the textbook. These parsing functions come almost directly (“mechanically”) from the productions of the grammar, as seen in class. You will demonstrate your parser by asking the user for input strings, parsing them, and printing the resulting parse tree (or an error message).

You must also implement a table-driven parser for the grammar, following the description seen in class and in the textbook, and demonstrate its use. Again: it’s not about being a creative programmer. It’s about understanding and applying the formal model. In this case, your code should be able to parse strings using **any** grammar given a parsing table connected to the productions of the grammar. But you will only need to show it working on one grammar.

Please read both the language description and the requirements carefully.

Language

Here is an informal English description of the language that we will be using this term:

1. Each string in the language consists of two parts.
2. The first part is either a `b` or it is one or more `c`'s.
3. The second part is an `e` enclosed in zero or more sets of parentheses.

Here are some of the strings in this language:

`be`, `c(e)`, `cc(e)`, `b((e))`

Here is a formal grammar for the language, parsable by a recursive descent or top-down table-driven parser:

$$\begin{aligned}\langle A \rangle &\rightarrow \langle B \rangle \langle E \rangle \\ \langle B \rangle &\rightarrow b \mid \langle C \rangle \\ \langle C \rangle &\rightarrow c \langle D \rangle \\ \langle D \rangle &\rightarrow \langle C \rangle \mid \epsilon \\ \langle E \rangle &\rightarrow (\langle E \rangle) \mid e\end{aligned}$$

Although there are an infinite number of other grammars that generate the same language, you **MUST** use **this** grammar for your project.

Requirements

Part 1: Recursive-descent parser (60%)

Implement a recursive-descent parser that produces parse trees for expressions of the required language.

- **You MUST use the grammar given in this document.**
- The style of the parsing functions **MUST** be as seen in class (which is sort of like what is in the textbook; check out the code available on BlackBoard with the project description).
- Your parsing functions **MUST** use functions `lookahead` and `match` as seen in class. Take a look at the code posted with the project description.
- You should be able to create the parsing functions by reading the productions of the grammar with almost no thinking required.
- The only place where thinking is required is how to use the lookahead symbol to choose productions, as seen in class and in the textbook.
- This will be boring if you do it right. I'm sorry.

You must then demonstrate your parser by reading strings from the user, parsing them, and printing the resulting parse tree (a so-called “read-eval-print loop” or “REPL”).

- Your program must prompt the user for input.
- You may assume that expressions are on a single line of input, and no longer than 255 characters. Note that the empty string is a valid input, so you should use `fgets` rather than `scanf` to read a line of input. See the “C for Java Programmers” guide and/or ask in study session if you need help with this.
- **Do not use `gets`. It is not legal in C11.**
- If the input is not well-formed, your parser should print an error message and resume processing at the next line of input.
- Otherwise print the parse tree as described below.
- This phase of the program should terminate when it reads the string “quit” (without the quotes).

Part 2: Table-driven parser (40%)

Implement a table-driven parser for expressions of the required language.

- As for Part 1: read expressions from the user, try to parse the input, print the parse tree or an error message, until “quit”.
- Most of the infrastructure of the parser will be the same as for Part 1.
- **You MUST use the grammar given in this document.**
- You **MUST** use an explicit parsing table that references explicitly-represented productions (FOCS Figs. 11.31 and 11.32). That means the table must contain either indexes of productions in a list or array of productions (as seen in the textbook), or references (pointers) to the productions themselves. Note: This does not mean pointers to parsing functions. It means some representation of the productions: head syntactic category and body consisting of zero or more constituents (terminal symbols or syntactic categories).
- You **MUST** have a function that creates and returns an instance of this table for the grammar of the required language. This function does **not** need to **translate** a grammar into a parsing table. It just needs to build and return the parsing table for the grammar that you are using. So work it out by hand, then write the code to produce it.
- You **MUST** have a **single** parsing function that takes **a parsing table, the start symbol of the grammar, and an input string** as arguments and does a table-driven parse. (You may have helper functions also.) Note that this function should be able to parse using **any** grammar, given its parsing table, start symbol, and productions. That is, it is **independent** of the grammar that it is using (but you only need to show it working with the required grammar).
- It may be helpful to produce output like FOCS Fig. 11.34 during debugging.

Your project **MUST** be a single program. This program should read the input and then call each parser (assuming you do both parts) and print the results, as described above. It is **your responsibility** to make it clear to us what your program is doing.

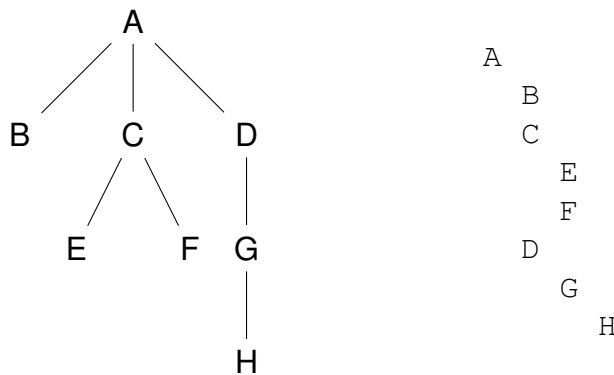


Figure 1: Example parse tree and corresponding printed output

Parse Trees and Printing Parse Trees

A parse tree is a dynamic data structure. You have seen trees in Java and they're the same in C. The textbook has an entire chapter on trees (Chapter 5), and the chapter for this unit has useful code also (also available on BlackBoard with the project description).

For this project, your program must print the resulting parse tree to standard output. There are many ways to do this, but for this project you will produce output in an *indented, pretty-printed format*. What this means is:

- Each node is printed on a separate line.
- The children of a node are indented relative to their parent.
- All children of a node are at the same level of indentation.
- The empty string symbol epsilon “ ϵ ” can be printed in C using by using a Unicode escape: “`\u03B5`”. If that doesn't work for you, just print the word “empty”.

Printing a tree involves doing a tree traversal, right? Traversing a tree is a recursive procedure, right? You print nodes and you recursively print their children, in the right order. So you can implement it using a recursive function. It's elegant *and* practical.

You also need to keep track of the current indentation level. So this will be a parameter to your pretty-printing function. In C, which does not have function overloading, this usually means two functions: a toplevel pretty-print function with no indentation parameter, and a helper function with that parameter, called from the toplevel function with indentation 0 to get the ball rolling.

If this did not make sense to you, please get to study session ASAP.

Additional Requirements and Policies

The short version:

- You **must** use the following C compiler options:
`-std=c11 -Wall -Werror`
- If you are using an IDE, you **must** configure it to use those options (but I suggest that you take this opportunity to learn how to use the command-line).
- You **must** submit a ZIP including your source code and a README by the deadline.
- You **must** tell us how to build your project in your README.
- You **must** tell us how to run your project in your README.
- Projects that do not compile will receive a grade of **0**.
- Projects that do not run or that crash will receive a grade of **0** for whatever parts did not work.
- Late projects will receive a grade of **0** (see below regarding extenuating circumstances).
- You will learn the most if you do the project yourself, but collaboration is permitted in teams of up to three (3) students.
- Do not copy code from other students or from the Internet or use generative AI to do your work for you.

Detailed information follows. . .

Programming Requirements

C programs **must** be written using the “C11” dialect of C. This means using the “`-std=c11`” option with `gcc` or `clang`. For more information, see [Wikipedia](#).

You **must** also use the options “`-Wall -Werror`”. These cause the compiler to report all warnings, and to make any warnings into errors that prevent your program from compiling. You **must** be able to write code without warnings in this course.

With these settings, your program should compile and run consistently on any platform. We will deal with platform-specific discrepancies as they arise.

If you are using an IDE (Eclipse, XCode, VSCode, CLion, *etc.*), you **must** ensure that it will also build as described above. The easiest way to do that is to setup the IDE with the required compiler options. There are some notes about this in the [C Programming Resources \(for CSC173 and beyond\)](#) area.

You may **NOT** use `#pragma`'s in your programs. This includes `#pragma`'s added by an IDE or any other way that they might get into your code.

You **SHOULD** test your program with the memory checking program `valgrind`. If you don't know what `valgrind` is or why it is A Good Thing, read [C for Java Programmers Chapter 11: Debugging a C Program](#). Mac users may also want to look into the `leaks` program, which is similar.

Programs that do not receive a clean report from `valgrind` have problems that **should be fixed** whether or not the program sometimes runs properly on some platforms. The only exception is unfreed memory errors (so-called “memory leaks”). We will not penalize you for those in CSC173. However if your program does not work for us, the first thing we're going to do is run `valgrind` on it. If `valgrind` reports errors: you have errors in your program. Period.

It is easy to run `valgrind` in a virtual machine or Docker container if you cannot install and run it natively. Mac and Windows users can install [Docker Desktop](#). See [How to run Linux \(including via Docker\)](#) for more information about Linux and Docker.

For help with `valgrind`, please go to study session **well before** the project deadline.

Submission Requirements

You **must** submit your project as a ZIP archive of a folder (directory) containing the following items:

1. A file named `README.txt` or `README.pdf` (see below)
2. The source code for your project (do not include object files or executables in your submission)
3. A completed copy of the submission form posted with the project description.

The name of the folder in ZIP **must** include “CSC173”, “Project 1” (or whatever), and the NetID(s) of the submitters. For example: “CSC173_Project_1_aturing”

Your README **must** include the following information:

1. The course: “CSC173”
2. The assignment or project (e.g., “Project 1”)
3. Your name and email address
4. The names and email addresses of any collaborators (per the course policy on collaboration)
5. Instructions for building your project (with the required compiler options)
6. Instructions for running your project

The purpose of the submission form is so that we know which parts of the project you attempted and where we can find the code for some of the key required features.

- **Projects without a submission form or whose submission form does not accurately describe the project will receive a grade of 0.**
- If you cannot complete and save a PDF form, submit a text file containing the questions and your (brief) answers.

Project Evaluation

You **must** tell us in your README how to build your project and how to run it.

Note that we will NOT load projects into Eclipse or any other IDE. We **must** be able to build and run your programs from the command-line. If you have questions about that, go to a study session.

We **must** be able to cut-and-paste from your documentation in order to build and run your code. **The easier you make this for us, the better your grade will be.** It is **your** job to make the building of your project easy and the running of its program(s) easy and informative.

For C projects, the most common command for building a program from all the C source files in the directory (folder) is:

```
gcc -std=c11 -Wall -Werror -o EXECUTABLE *.c
```


where `EXECUTABLE` is the name of the executable program that we will run to execute your project.

You may also tell use to build your project using `make`. In that case, be sure to include your `Makefile` with your submission. You **must** ensure that your `Makefile` sets the compiler options appropriately.

If you expect us to do something else, you **must** describe what we need to do in your `README` file. This is unlikely to be the case for most of the projects in CSC173.

Please note that we will **NOT** under any circumstances edit your source files. That is your job.

Projects that do not compile will receive a grade of 0. There is no way to know if your program is correct solely by looking at its source code (although we can sometimes tell that is incorrect). This is actually an aspect of a very deep result in Computer Science that we cover in CSC173.

We will then run your program by running the executable, or as described in the project description. If something else is required, you **must** describe what is needed in your `README` file.

Projects that do not run or that crash will receive a grade of 0 for whatever parts did not work. You earn credit for your project by meeting the project requirements. Projects that do not run **do not** meet the requirements.

Any questions about these requirements: go to study session **BEFORE** the project is due.

Late Policy

Late projects will receive a grade of 0. You **must** submit what you have by the deadline. If there are extenuating circumstances, submit what you have before the deadline and then explain yourself via email.

If you have a medical excuse (see the course syllabus), submit what you have and explain yourself as soon as you are able.

Collaboration Policy

I assume that you are in this course to learn. You will learn the most if you do the projects **yourself**.

That said, collaboration on projects is permitted, subject to the following requirements:

- Teams of no more than 3 students, all currently taking CSC173.
- You **must** be able to explain anything you or your team submit, IN PERSON AT ANY TIME, at the instructor's or TA's discretion.
- One member of the team should submit code on the team's behalf in addition to their writeup. Other team members **must** submit a README (only) indicating who their collaborators are.
- All members of a collaborative team will get the same grade on the project.

Working in a team only works if you actually do all parts of the project together. If you only do one part of, say, three, then you only learn one third of the material. If one member of a team doesn't do their part or does it incorrectly (or dishonestly), all members pay the price.

Academic Honesty

I assume that you are in this course to learn. You will learn nothing if you do not do the projects **yourself**.

Do not copy code from other students or from the Internet.

Avoid Github and StackOverflow completely for the duration of this course.

The use of generative AI tools is **NOT** permitted in this course.

Posting homework and project solutions to public repositories on sites like GitHub is a violation of the University's Academic Honesty Policy, Section V.B.2 "Giving Unauthorized Aid." Honestly, no prospective employer wants to see your coursework. Make a great project outside of class and share that instead to show off your chops.

A Example Transcript

Enter input string ("quit" to quit): cce

A
 B
 C
 c
 D
 C
 c
 D
 ϵ
E
 e

Enter input string ("quit" to quit): b(e)

A
 B
 b
 E
 (
 E
 e
)

Enter input string ("quit" to quit): quit